

10 — Teardown: Effect Cleanup

LIFECYCLE

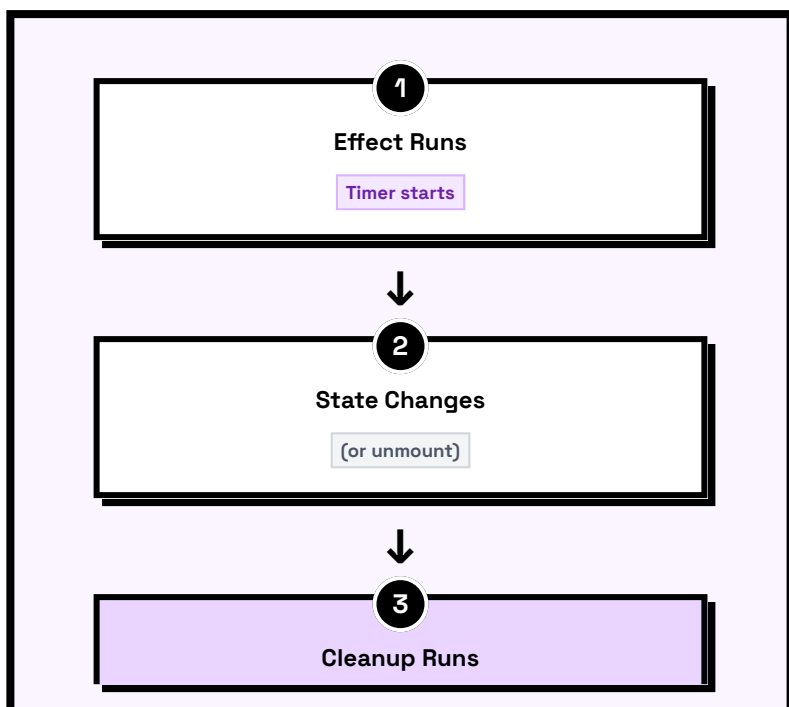
Q. How do you prevent memory leaks from timers or subscriptions created inside an `$effect`?

ANSWER. If you start a `setInterval` timer inside a component to build a stopwatch, that timer will keep firing forever—even if the user navigates to a totally different page, severely lagging the browser. You return a cleanup function from within the `$effect` block, guaranteeing the timer is destroyed the second the component leaves the screen.

Timer.svelte

```
<script>
  $effect(() => {
    const timer = setInterval(() =>
      console.log('tick'), 1000);

    // Return a function to clean up
    return () => clearInterval(timer);
  });
</script>
```



Timer stops

- ① **Summary.** Return a function from your **\$effect** to clean up tasks. This is important because it guarantees that existing timers or data subscriptions are safely destroyed when the component unmounts. This prevents massive memory leaks with zero extra boilerplate.